PYTHON APPLICATION PROGRAMMING

Anitya Kumar Gupta

Errors and Exceptions

Aspect	Errors	Exceptions
Origin	Errors are typically caused by the environment, hardware, or operating system.	Exceptions are usually a result of problematic code execution within the program.
Nature	Errors are often severe and can lead to program crashes or abnormal termination.	Exceptions are generally less severe and can be caught and handled to prevent program termination.
Handling	Errors are not usually caught or handled by the program itself.	Exceptions can be caught using try-except blocks and dealt with gracefully, allowing the program to continue execution.
Examples	Examples include "SyntaxError" due to incorrect syntax or "NameError" when a variable is not defined.	Examples include "ZeroDivisionError" when dividing by zero, or "FileNotFoundError" when attempting to open a non-existent file.
Categorization	Errors are not classified into categories.	Exceptions are categorized into various classes, such as "Arithmetic Error," "IOError," ValueError," etc., based on their nature.

Common Exceptions in Python

- **ZeroDivisionError:** This error arises when an attempt is made to divide a number by zero. Division by zero is undefined in mathematics, causing an arithmetic error.
- **ValueError:** This error occurs when an inappropriate value is used within the code.
- **FileNotFoundError:** This exception is encountered when an attempt is made to access a file that does not exist.
- **IndexError:** An IndexError occurs when an index is used to access an element in a list that is outside the valid index range.
- **KeyError:** The KeyError arises when an attempt is made to access a non-existent key in a dictionary.
- **TypeError:** The TypeError occurs when an object is used in an incompatible manner.
- **AttributeError:** An AttributeError occurs when an attribute or method is accessed on an object that doesn't possess that specific attribute or method.
- **ImportError:** This error is encountered when an attempt is made to import a module that is unavailable.

Examples

Run each piece of code and observe the exception raised

[1]:
```
1/0
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[1], line 1
----> 1 1/0

ZeroDivisionError: division by zero
```

`ZeroDivisionError` occurs when you try to divide by zero.

[2]:
```
y = a + 5
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 y = a + 5

NameError: name 'a' is not defined
```

`NameError` -- in this case, it means that you tried to use the variable a when it was not defined.

[3]:
```
a = [1, 2, 3]
a[10]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[3], line 2
      1 a = [1, 2, 3]
----> 2 a[10]

IndexError: list index out of range
```

`IndexError` -- in this case, it occured because you tried to access data from a list using an index that does not exist for this list.

Try and Except

Here's how they work:

1. The code that may result in an exception is contained in the try block.
2. If an exception occurs, the code directly jumps to except block.
3. In the except block, you can define how to handle the exception gracefully, like displaying an error message or taking alternative actions.
4. After the except block, the program continues executing the remaining code.

Example

[2]:
```
a = 1

try:
    b = int(input("Please enter a number to divide a"))
    a = a/b
    print("Success a=",a)
except:
    print("There was an error")
```

```
Please enter a number to divide a 5
Success a= 0.2
```

Try Except Specific

- A specific try except allows you to catch certain exceptions and also execute certain code depending on the exception.
- This is useful if you do not want to deal with some exceptions and the execution should halt.
- It can also help you find errors in your code that you might not be aware of.
- Furthermore, it can help you differentiate responses to different exceptions. In this case, the code after the try except might not run depending on the error.

Example

This is the same example as above, but now we will add differentiated messages depending on the exception, letting the user know what is wrong with the input.

[22]:
```
a = 1

try:
    b = int(input("Please enter a number to divide a"))
    a = a/b
    print("Success a=",a)
except ZeroDivisionError:
    print("The number you provided cant divide 1 because it is 0")
except ValueError:
    print("You did not provide a number")
except:
    print("Something went wrong")
```

```
Please enter a number to divide a 65
Success a= 0.015384615384615385
```